

Krypton

Migración de un Monolito a Microservicios

CIBERTEC · Desarrollo de Aplicaciones Web II — EFSRT VI

Tayra · Jason · Curi · Cristofer · Sergio

Agenda

1. Qué es Krypton
2. Por qué microservicios
3. La arquitectura
4. Cada servicio y patrón
5. Demo en vivo
6. Conclusiones

El proyecto Krypton

- E-commerce B2C de tecnología
- Monolito **Spring Boot 3 + Java 17**
- Frontend **React 19 · MySQL 8**
- Catálogo, carrito, checkout, órdenes, admin
- Ya funcionaba: **+400 tests verdes**

¿Por qué microservicios?

- Monolito: **todo en un proceso**
- No escala por partes · un fallo afecta todo
- Despliegue acoplado
- → cada parte **independiente**, su base, escala sola
- Patrones: **Eureka · Feign · RabbitMQ · Docker**

Arquitectura general

- Frontend → **api-gateway (:8080)**
- 9 servicios, todos en **Eureka (:8761)**
- **1 base MySQL por servicio** + RabbitMQ
- 3 ideas: 1 base/servicio · se llaman por **nombre** · **JWT = pasaporte**



Visual: insertar acá el diagrama de arquitectura (README).

Service Discovery — Eureka

- Las IPs/puertos cambian: **no se hardcodean**
- Eureka = **registro vivo** (la "guía telefónica")
- Cada servicio se anota al arrancar
- Los demás lo buscan **por nombre**

API Gateway

- Una **sola puerta** (:8080)
- Rutea por nombre: lb://users-service
- Resuelve con Eureka + **balancea**
- Centraliza CORS · el front no sabe los puertos

Dockerización

- Un **Dockerfile** por servicio
- `docker compose up` → **11 contenedores**
- Parametrizado por **env**: `localhost` → nombre de contenedor

Login: el flujo

- Front → POST /api/auth/login
- **users** verifica password (BCrypt)
- **Emite el JWT** → el front lo guarda
- Se manda en **cada request**

¿Qué es un JWT?

- `header.payload.signature`
- `header = algoritmo · payload = email + rol + exp`
- `signature = firma`
- **Firmado, NO encriptado** (se lee, no se modifica)

JWT stateless

- **users FIRMA** con el secreto
- los demás **VERIFICAN** con el MISMO secreto
- sin sesión, sin DB, sin preguntar a users
- secreto compartido = el "**contrato**"

catalog-service

- Dueño del catálogo: **productos, categorías, stock**
- base `krypton_catalog`
- Valida un JWT que **NO emitió** (stateless)
- Expone el **stock** para el checkout

review-service — Reseñas

- Rating **1-5** + comentario · base `krypton_reviews`
- Promedio con **AVG**
- Valida que el producto exista **vía Feign** → **catalog**
- 1 reseña por usuario por producto

El problema del checkout

- El checkout debe **descontar stock**
- ...pero el stock vive en **catalog** (otra base, otro proceso)
- **No hay** @Transactional entre servicios
- ¿Cómo garantizo **"todo o nada"**?

Feign + Saga con compensación

- **Feign** = llamar a otro servicio como un método (síncrono)
- order pide a catalog: **descontar** stock
- Si algo falla después → **compensación**
- order pide a catalog: **restaurar** stock (rollback a mano)

Cupón de descuento

- `couponCode` en el checkout
- order pide el descuento a **promo** (Feign)
- `total = subtotal - descuento + envío`
- El descuento es **REAL**: lo decide el backend

RabbitMQ — Mensajería asíncrona

- Síncrono (Feign) = **llamada telefónica**
- Asíncrono (RabbitMQ) = **WhatsApp**
- order **publica** "orden creada" → notification **consume**
- Desacople total

payment-service

- POST /api/orders/{id}/pay
- order delega el cobro a **payment** (Feign)
- **Máquina de estados**: PENDIENTE → CONFIRMADA
- 402 si se rechaza · 422 si la transición no vale

promo-service

- Cupones: **PORCENTAJE** o **MONTO**
- Admin **crea / lista**
- `POST /api/promos/apply` calcula el descuento
- base `krypton_promos`

Frontend conectado

- **React 19 + Vite** · todo pega al gateway (:8080)
- Nuevo: **reseñas** en el producto
- **Cupón** en el checkout · **pago** de la orden
- **Panel admin** de cupones

Calidad y verificación

- TDD donde aporta: **saga, stock, pagos, cupón** (verdes)
- Verificación con **curls reales** + chequeo en DB
- Todo **containerizado**

Demo en vivo

Login → Catálogo → Reseña → Carrito → Checkout con cupón → Pagar → Notificación

Conclusiones y aprendizajes

- Patrones: discovery, gateway, **JWT stateless**, **Feign + saga**, RabbitMQ, Docker
- Trade-offs: stateless = rápido, pero sin baja inmediata
- Gotchas resueltos (RabbitMQ en Docker, el 401 del error-dispatch)

¡Gracias!

Código y guías en **GitHub**

¿Preguntas?